

AIM:

- To implement and train a Single Shot Detector (SSD) network for detecting objects relevant to self-driving cars (like pedestrians, traffic lights, and vehicles).
- To use object detection to enhance environment awareness in autonomous driving.

Algorithm:

1. **Import Libraries:** Use PyTorch, torchvision, and relevant modules for data and modeling.
2. **Load Dataset:** Use a dataset such as Pascal VOC or a custom self-driving dataset.
3. **Preprocess Data:** Resize images, normalize, and apply augmentations.
4. **Define SSD Model:** Load pre-trained SSD (e.g., SSD300) from `torchvision.models.detection`.
5. **Modify for Custom Classes:** If needed, change the number of output classes.
6. **Set Loss Function and Optimizer:** Use multi-box loss and Adam/SGD optimizer.
7. **Train the Model:** Feed images and annotations into the model and update weights.
8. **Evaluate and Visualize:** Predict bounding boxes and visualize results on test images.

Code:

```
# Load YOLO model
net = cv2.dnn.readNet("yolov4.weights", "yolov4.cfg")
layer_names = net.getLayerNames()
output_layers = [layer_names[i - 1] for i in
net.getUnconnectedOutLayers()]
classes = open("coco.names").read().strip().split("\n")

# Load the video
```

```

video_path = "/content/Video.mp4" # Change to your video file
cap = cv2.VideoCapture(video_path)

# Get original video properties
frame_width = int(cap.get(3)) # Original width
frame_height = int(cap.get(4)) # Original height
fps = int(cap.get(5)) # Original frame rate

# Video writer (same resolution & FPS as input)
out = cv2.VideoWriter("output.avi", cv2.VideoWriter_fourcc(*'XVID'), fps,
(frame_width, frame_height))

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break # Stop if video ends

    height, width, _ = frame.shape # Keep original dimensions

    # Convert frame to blob (without rescaling)
    blob = cv2.dnn.blobFromImage(frame, 1/255.0, (416, 416), swapRB=True,
crop=False)
    net.setInput(blob)
    detections = net.forward(output_layers)

    boxes, confidences, class_ids = [], [], []

    # Process detections
    for output in detections:
        for detection in output:
            scores = detection[5:]
            class_id = np.argmax(scores)
            confidence = scores[class_id]

            if confidence > 0.5:
                center_x, center_y, w, h = map(int, detection[:4] *
[width, height, width, height])
                x, y = int(center_x - w / 2), int(center_y - h / 2)
                boxes.append([x, y, w, h])
                confidences.append(float(confidence))

```

```

        class_ids.append(class_id)

# Apply Non-Maximum Suppression (NMS)
indexes = cv2.dnn.NMSBoxes(boxes, confidences, 0.5, 0.4)

# Draw bounding boxes (keeping original frame size)
for i in indexes.flatten():
    x, y, w, h = boxes[i]
    label = f"{classes[class_ids[i]]}: {confidences[i]:.2f}"
    color = (0, 255, 0)
    cv2.rectangle(frame, (x, y), (x + w, y + h), color, 2)
    cv2.putText(frame, label, (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX,
0.5, color, 2)

# Save and display the frame without resizing
out.write(frame)
cv2.imshow(frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break # Press 'q' to stop

# Release resources
cap.release()
out.release()
cv2.destroyAllWindows()

```

Output:



Result:

- Successfully implemented an SSD-based object detector for self-driving car scenarios.
- The model detected key objects like cars and pedestrians from input images using real-time inference.